PUC Minas – Coração Eucarístico
Engenharia de Software
Disciplina: Teste de Software

Detecção de Bad Smells e Refatoração Segura

Aluno: João Gabriel Maia da Costa
Matrícula: 837351

1. Análise de Smells

Analisando manualmente `src/ReportGenerator.js`, encontrei três Bad Smells:

(a) Método Longo / Complexidade Ciclomática alta. `generateReport` mistura cabeçalho, corpo, rodapé, filtro por perfil e formatação em um único método, com ifs aninhados em até 4 níveis. É difícil de ler, e qualquer mudança (um novo formato, um novo perfil) obriga a editar o centro do método e retestar todas as combinações.

(b) Código Duplicado. A linha de CSV `` `${item.id},${item.name},...` `` e o `total += item.value` aparecem em quatro ramos (ADMIN/USER × CSV/HTML). Corrigir um bug ou mudar o formato exige lembrar de todas as cópias; os testes precisam cobrir cada cópia separadamente.

(c) Números Mágicos e efeito colateral. Os valores `1000` e `500` aparecem sem nome, escondendo a regra de negócio. Além disso, o método altera o objeto recebido (`item.priority = true`), um efeito colateral oculto que torna os testes dependentes do estado dos dados. Há também um bug visível: para itens sem prioridade o HTML sai como `<tr >` (espaço extra), e por isso 1 dos 5 testes originais já falha antes de qualquer refatoração.

2. Relatório da Ferramenta

Saída de `npx eslint src/` antes da refatoração (com `sonarjs/cognitive-complexity` limite 5, conforme o `.eslintrc.json` do repositório):

```
src\ReportGenerator.js
  11:3   error  Refactor this function to reduce its Cognitive Complexity from 27 to the 5
allowed  sonarjs/cognitive-complexity
  43:14  error  Merge this if statement with the nested one
sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)
```

A análise manual apontou "método longo" de forma qualitativa. O SonarJS quantificou o problema: complexidade cognitiva **27** contra o limite de 5, um número objetivo que mostra a gravidade e permite comparar antes/depois. Ele também achou um `if` colapsável (linha 43) que passa despercebido na leitura. Por outro lado, a ferramenta não apontou os números mágicos nem o efeito colateral, que só a leitura humana pegou; as duas abordagens se complementam.

3. Processo de Refatoração

O smell mais crítico é a complexidade do `generateReport`. **Antes** (trecho):

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) { item.priority = true; }
    if (reportType === 'CSV') { report += `${item.id},...\n`; total += item.value; }
    else if (reportType === 'HTML') { ...; total += item.value; }
```

```
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') { ... } else if (reportType === 'HTML') { ... }
    }
  }
}
```

Depois:

```
generateReport(reportType, user, items) {
  const formatter = formatters[reportType] ?? emptyFormatter;
  const visibleItems = this.filterVisibleItems(user, items);
  const header = formatter.header(user);
  const body = visibleItems
    .map((item) => formatter.row(item, user, isPriority(user, item))).join('');
  const total = this.calculateTotal(visibleItems);
  return (header + body + formatter.footer(total)).trim();
}
```

Técnicas aplicadas: **Extract Method** (`filterVisibleItems`, `calculateTotal`, `isPriority`); **Replace Conditional with Polymorphism** (objetos `csvFormatter` e `htmlFormatter` com a mesma interface `header/row/footer` e uma tabela de políticas de visibilidade por perfil, no lugar dos if/else de tipo e role); **Replace Magic Number with Constant** (`ADMIN_PRIORITY_THRESHOLD`, `USER_MAX_VALUE`); e remoção do efeito colateral (a prioridade é calculada, não gravada no item). A duplicação desapareceu: o total é calculado uma única vez e cada formato é definido em um só lugar. Adicionar um novo formato agora exige apenas um novo objeto formatador.

Resultado: `npx eslint src/ReportGenerator.refactored.js` não reporta nenhum erro (complexidade 27 → abaixo de 5). Todos os 5 testes passam contra o arquivo refatorado.

4. Validação com a Suíte de Testes

Os testes originais não foram alterados. Criei `__tests__/ReportGenerator.refactored.test.js`, cópia do original trocando apenas o import. Resultado de `npm test`: o arquivo refatorado passa em 5/5. O teste original falha em 1 caso (HTML de Admin) por causa do bug do `<tr >` descrito acima; a refatoração corrige esse bug. Para executar os testes em ESM foi necessário ajustar o script `test` do `package.json` para `node --experimental-vm-modules node_modules/jest/bin/jest.js`.

5. Conclusão

A suíte de testes foi essencial: como ela descreve o comportamento (saída) e não a implementação, pude reescrever a estrutura inteira e verificar a cada passo que o resultado continuava correto. Sem ela, trocar if/else por formatadores seria um salto no escuro. Reduzir smells diminuiu a complexidade cognitiva de 27 para um nível baixo, eliminou duplicação e deixou o código mais fácil de ler, estender e testar em unidades pequenas, o que reduz o custo e o risco de manutenções futuras.